LAB – 2

Write a C program to simulate SJF (Preemptive and Non-Preemptive) and Priority Scheduling (Preemptive and Non-Preemptive) algorithms

```
#include <stdio.h>

struct Process

{

    int pid, at, bt, ct, wt, tat, priority, rt, completed;

};


void original(struct Process a[], struct Process b[], int n)

{

    for(int i=0;i<n;i++)

        b[i] = a[i];

}


void result(struct Process p[], int n)

{

    float avgwt=0,avgtat=0;

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

    for(int i=0;i<n;i++)

    {

        p[i].tat = p[i].ct - p[i].at;

        p[i].wt = p[i].tat - p[i].bt;

        avgwt += p[i].wt;

        avgtat += p[i].tat;
```

```
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,p[i].at,p[i].bt,p[i].ct,p[i].tat,p[i].wt);
    }
    printf("Average WT = %.2f\n",avgwt/n);
    printf("Average TAT = %.2f\n",avgtat/n);
}
void sjf_np(struct Process p[], int n)
{
    printf("\n--- SJF NON PREEMPTIVE ---\n");
    int time=0,completed=0,min,pos;
    while(completed<n)
    {
        min=9999;
        pos=-1;
        for(int i=0;i<n;i++)
        {
            if(p[i].at<=time && p[i].completed==0 && p[i].bt<min)
            {
                min=p[i].bt;
                pos=i;
            }
        }
        if(pos!=-1)
        {
            time+=p[pos].bt;
            p[pos].ct=time;
            p[pos].completed=1;
```

```
            completed++;
        }
        else
            time++;
    }
    result(p,n);
}
void sjf_p(struct Process p[], int n)
{
    printf("\n--- SJF PREEMPTIVE ---\n");
    int time=0,completed=0,min,pos;
    while(completed<n)
    {
        min=9999;
        pos=-1;
        for(int i=0;i<n;i++)
        {
            if(p[i].at<=time && p[i].rt>0 && p[i].rt<min)
            {
                min=p[i].rt;
                pos=i;
            }
        }
        if(pos!=-1)
        {
            p[pos].rt--;
            time++;
```

```
            if(p[pos].rt==0)
            {
                p[pos].ct=time;
                completed++;
            }
        }
        else
            time++;
    }
    result(p,n);
}
void priority_np(struct Process p[], int n)
{
    printf("\n--- PRIORITY NON PREEMPTIVE ---\n");
    int time=0,completed=0,high,pos;
    while(completed<n)
    {
        high=9999;
        pos=-1;
        for(int i=0;i<n;i++)
        {
            if(p[i].at<=time && p[i].completed==0 && p[i].priority<high)
            {
                high=p[i].priority;
                pos=i;
            }
        }
```

```
        if(pos!=-1)
        {
            time+=p[pos].bt;
            p[pos].ct=time;
            p[pos].completed=1;
            completed++;
        }
        else
            time++;
    }
    result(p,n);
}
void priority_p(struct Process p[], int n)
{
    printf("\n--- PRIORITY PREEMPTIVE ---\n");
    int time=0,completed=0,high,pos;
    while(completed<n)
    {
        high=9999;
        pos=-1;
        for(int i=0;i<n;i++)
        {
            if(p[i].at<=time && p[i].rt>0 && p[i].priority<high)
            {
                high=p[i].priority;
                pos=i;
            }
```

```
        }
        if(pos!=-1)
        {
            p[pos].rt--;
            time++;
            if(p[pos].rt==0)
            {
                p[pos].ct=time;
                completed++;
            }
        }
        else
            time++;
    }
    result(p,n);
}
int main()
{
    int n;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    struct Process p[n], temp[n];
    for(int i=0;i<n;i++)
    {
        printf("\nProcess %d\n",i+1);
        p[i].pid=i+1;
        printf("Arrival Time: ");
```

```
        scanf("%d",&p[i].at);
        printf("Burst Time: ");
        scanf("%d",&p[i].bt);

        printf("Priority: ");
        scanf("%d",&p[i].priority);
        p[i].rt=p[i].bt;
        p[i].completed=0;
    }
    original(p,temp,n);
    sjf_np(temp,n);

    original(p,temp,n);
    sjf_p(temp,n);

    original(p,temp,n);
    priority_np(temp,n);

    original(p,temp,n);
    priority_p(temp,n);

    return 0;
}
```

OUTPUT :

```
Enter number of processes: 5

Process 1
Arrival Time: 0
Burst Time: 3
Priority: 5

Process 2
Arrival Time: 2
Burst Time: 2
Priority: 3

Process 3
Arrival Time: 3
Burst Time: 5
Priority: 2

Process 4
Arrival Time: 4
Burst Time: 4
Priority: 4

Process 5
Arrival Time: 6
Burst Time: 1
Priority: 1

--- SJF NON PREEMPTIVE ---

PID     AT      BT      CT      TAT     WT
P1      0       3       3       3       0
P2      2       2       5       3       1
P3      3       5       15      12      7
P4      4       4       9       5       1
P5      6       1       10      4       3
Average WT = 2.40
Average TAT = 5.40
```

```
--- SJF PREEMPTIVE ---

PID     AT      BT      CT      TAT     WT
P1      0       3       3       3       0
P2      2       2       5       3       1
P3      3       5       15      12      7
P4      4       4       10      6       2
P5      6       1       7       1       0
Average WT = 2.00
Average TAT = 5.00

--- PRIORITY NON PREEMPTIVE ---

PID     AT      BT      CT      TAT     WT
P1      0       3       3       3       0
P2      2       2       11      9       7
P3      3       5       8       5       0
P4      4       4       15      11      7
P5      6       1       9       3       2
Average WT = 3.20
Average TAT = 6.20

--- PRIORITY PREEMPTIVE ---

PID     AT      BT      CT      TAT     WT
P1      0       3       15      15      12
P2      2       2       10      8       6
P3      3       5       9       6       1
P4      4       4       14      10      6
P5      6       1       7       1       0
Average WT = 5.00
Average TAT = 8.00
```